Site Web de l'association sportive L'amicale du Val de Somme

Version	Date	Auteur
2024	01/09/2024	Guy Verghote

Table des matières

1. La base de données 1
   1.1. Les données concernant les résultats des épreuves des 4 saisons 1
   1.2. La gestion des membres et des administrateurs 1
   1.3. Les données informatives 2
2. Source et structure de l'application 3
3. La structure de chaque fonctionnalité 4
4. La déclaration de type 5
5. L'accès aux données 5
6. Les classes techniques 6
7. Les composants 6
8. Les composants externes ..... **Erreur ! Signet non défini.**
9. Les librairies chargées via composer 7
10. Le traitement des erreurs 8
11. Les classes métiers 8
12. Contrôle d'accès sur le répertoire backoffice 11
13. La page d'accueil 12
14. Les pages consultables par tous les visiteurs. 13
15. Contrôle d'accès sur les fonctionnalités accessibles uniquement par les membres 14
16. Contrôle d'accès sur les fonctionnalités d'administration 14
17. La connexion 16
18. La modification des informations du profil 18
19. L'annuaire des membres 19
20. Les modules d'administration 20
21. La gestion des pages de présentation 21
22. La gestion des membres 22
23. L'ajout d'un membre 22
24. La planification des 4 saisons 23
25. La photothèque 24
26. La partie backoffice 25
27. Gestion des droits des administrateurs 25

1. La base de données

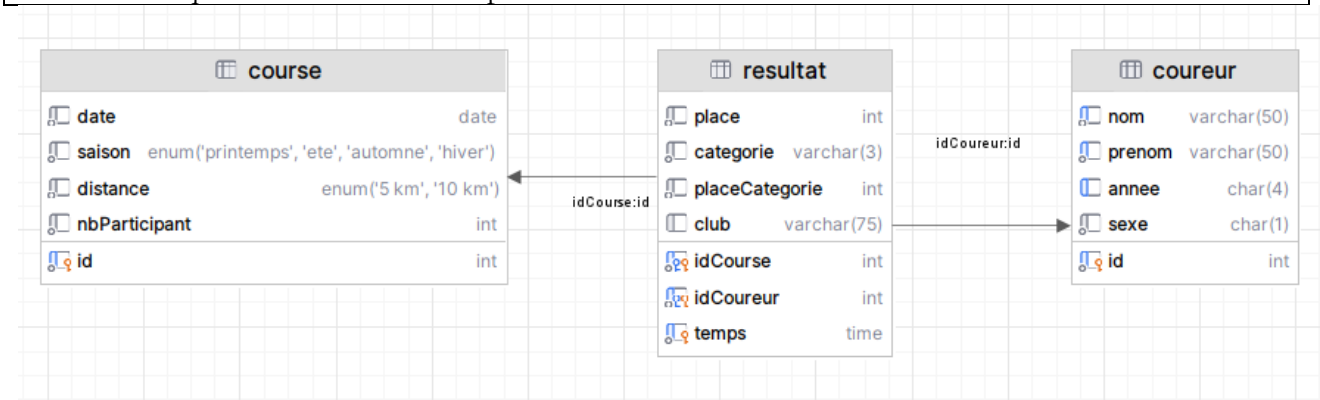
1.1. Les données concernant les résultats des épreuves des 4 saisons

L'association organise depuis 1983, 4 éditions d'une course appelée "Les 4 saisons"
Jusqu'en 2003 une seule distance de 10 km était proposée.
Depuis 2004, deux distances sont proposées : 5 km et 10 km.
On compte actuellement 220 courses disputées par 10841 coureurs soit 44215 résultats

Table	coureur
Rôle	Les coureurs ayant participé aux 4 saisons depuis l'origine
Champs	**id,** nom, prenom, annee, sexe
Remarque	Les données sont partielles, on ne connait pas toujours l'année. Un index unique est défini sur le triplet nom , prénom et année

Table	course
Rôle	Les courses des 4 saisons depuis l'origine
Champs	**id,**date, saison, distance, nbParticipant

Table	resultat
Rôle	Contient les performances de chaque participant à une épreuve des 4 saisons
Champs	i**dCourse**, place, **idCoureur**, categorie, placeCategorie, **temps**, club
Remarque	Le temps fait ici partie de la clé primaire car dans le cadre des 4 saisons le résultat de la course des 10 Km fusionne les résultats de deux courses (10 Km populaire + 45' et 10 km as – 45') et un coureur peut participer à ces deux courses. Le seul souci serait alors qu'il réalise le même temps sur ces deux courses !



1.2. La gestion des membres et des administrateurs

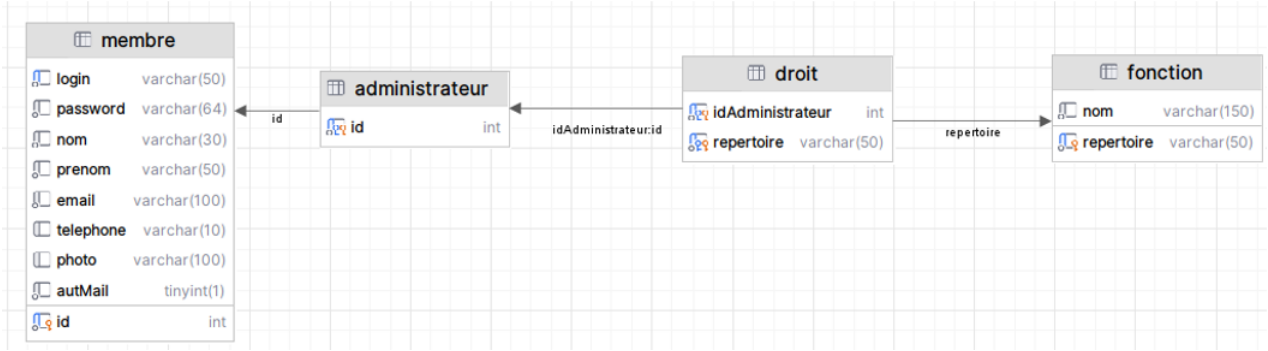
Table	Membre
Rôle	Contient les coordonnées sur les membres.
Champs	**id,** login, password, nom, prenom, email, telephone, photo, autMail
Remarque	index unique sur nom, prenom index unique sur login fixe, mobile, photo sont optionnels. Ils pourront être renseignés par le membre autMail permet de savoir si le membre autorise l'affichage de son email dans l'annuaire des membre

Parmi les membres se trouve des administrateurs qui peuvent réaliser un certain nombre de tâches administratives.

Table	administrateur
Rôle	Contient les id des membres définis comme administrateur
Champs	**id**
Liaison	avec la table **membre** : suppression et modification en cascade

Table	droit
Rôle	Associe les fonctions à un administrateur
Champs	**idAdministrateur, repertoire**
Liaison	avec la table **administrateur** et la table **Fonction :** suppression et modification en cascade pour les 2

Table	fonction
Rôle	Contient les fonctions d'administration
Champs	**repertoire** nom
Liaison	Avec la table **droit :** suppression en cascade



1.3. Les données informatives

Table	Page
Rôle	Contient les textes affichés dans les pages modifiables du site au nombre de 6 : Présentation club, adhésion au club, Présentation 4 saisons, Formation, Mentions légales, Politique de confidentialité
Champs	**id** nom, contenu
Remarque	L'ajout n'est pas possible depuis le site : un ajout nécessite d'ajouter une option dans le menu

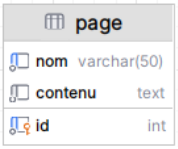
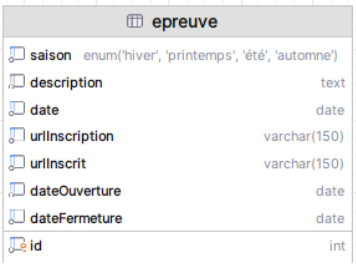


Table	Epreuve
Rôle	Contient les informations sur les 4 éditions des 4 saisons
Champs	**id** description, date, urlInscription, urlInscrit, dateOuverture dateFermeture
Remarque	Seule la modification des informations est possible. Les 'url' indique le site permettant de s'inscrire en ligne et de voir les inscrits. Les inscriptions sont ouvertes à une date donnée et fermées quelques jours avant la date de l'épreuve.



2. Source et structure de l'application

Répertoire	Contenu
.config	Fichiers de configuration spécifique à l'application pour la base de données, les journaux, etc.
.sql	Scripts pour la mise en place de la base de données vds2025
.powershell	Scripts automatisant les commandes git pour récupérer depuis le référentiel GitHub ou envoyer vers le référentiel GitHub les mises à jour.
4saisons	Module affichant la page de présentation des 4 saisons (page 2)
allure	Module du cadre 'Informations utiles pour tous les coureurs' qui affiche les temps de passage en fonction de son allure au kilomètre.
administration	Module affichant les fonctions d'administration accessibles par le membre connecté Ce répertoire contient un sous-répertoire pour chaque fonction d'administration réalisée ou à réaliser : classement, document, epreuve, evenement, information, membre, page, partenaire, photoinformation
ajax	Scripts PHP standards permettant l'ajout, la modification et la suppression dans une table. Ces scripts sont appelés par la méthode Ajax
backoffice	Module permettant l'administration technique du site. Ce répertoire contient un sous répertoire pour chaque fonctionnalité administrative technique : pour l'instant seule la gestion des droits est réalisée/ Son accès est protégé par un fichier .htaccess associé à un fichier .htpassword
classemetier	Classes PHP permettant de gérer et des consulter les données de la base de données. En règle générale, il existe une classe pour chaque table. La classe Membre qui gère les membre de l'association utilise le fichier de configuration configmembre.php afin de définir les paramètres que doit vérifier le fichier image contenant la photo du membre (extension, type mime, taille, etc.)
classetechnique	Classes PHP utilisables dans tous les projets (erreur, std, input et classes dérivées, table, select, fichier, etc.). Ce répertoire est en liaison avec le référentiel Github https://github.com/verghote/classetechnique.git
club	Page affichant les informations sur le club stockées dans la table page
composant	Scripts HTML contenant les balises link et script permettant le chargement des composants Javascript standard à partir de leur CDN. Contient aussi le code source d'une version personnalisée du composant ckeditor
css	Feuille de style et images associées
data	Contient tous les fichiers qui sont générés ou déposés par l'application (fichier PDF, fichier image, fichiers log etc.). Ces ressources sont classées dans des sous-répertoires : classement, document, partenaire (logo), photomembre, photoinformation
erreur	Scripts gérant toutes les erreurs détectées par le serveur Apache (401, 403, 404, 410) et les erreurs détectées par l'application
img	Images utilisées par l'application
include	Scripts PHP à inclure dans toutes les pages : autoload, interface (header, footer). Scrips PHP à inclure assurant la protection des accès
membre	Ensemble des modules accessibles aux membres (connexion, déconnexion, annuaire, modification de son profil, modification de son mot de passe, oubli du mot de passe)
mentionslegales	Module affichant les mentions légales (page 3)
politique	Module affichant la politique de confidentialité (page 4)

3. La structure de chaque fonctionnalité

Elle se compose de trois fichiers :

- Un fichier PHP qui fait office de contrôleur : il prend en charge le contrôle d'accès, le chargement des données et le transfert de ces données côté client dans des variables Javascript
- Un fichier HTML qui représente la vue statique (l'interface de la fonctionnalité)
- Un fichier .js qui représente la vue dynamique (affichage des données, procédures événementielles etc)

Le fichier PHP respecte la structure suivante :

```
// activation du chargement dynamique des ressources
require $_SERVER['DOCUMENT_ROOT'] . "/include/autoload.php";

// chargement des données
$ligne = Page::getPage(1);
$titreFonction = $ligne['nom'];
$data = json_encode($ligne);

// transmission des données à l'interface
$head = <<<EOD
   <script>
      let data = $data
   </script>
EOD;

// chargement des composants spécifiques (si besoin)
$head .= file_get_contents(RACINE . '/composant/autocomplete.html');

// chargement de l'interface
require RACINE . "/include/interface.php";
```

Pour les fonctionnalités ayant besoin de récupérer dynamiquement des données, un répertoire ajax contient les scripts PHP réalisant la récupération des données et l'envoie de la réponse dans le format JSON.

Le chargement des données s'effectue toujours par l'appel d'une méthode d'une classe.
Ces classes sont stockées dans le répertoire classemetier.
Il n'y a donc aucune instruction d'accès aux données et donc de requête SQL dans les fichiers PHP d'un module. Ces instructions se trouvent uniquement dans les instructions des méthodes des classes métiers.

L'utilisation de la classe technique Select facilite l'interrogation de la base de données en proposant trois méthodes pour récupérer les données :

- getRows : pour récupérer plusieurs enregistrements
- getRow : pour récupérer un seul enregistrement
- getValue : pur récupérer une valeur

La constante RACINE permet d'accéder aux ressources (classes, scripts) en utilisant un chemin absolu. Cela permet d'appeler une classe de la même façon quelle que soit la position du script appelant dans l'arborescence du site.
En effet il n'est pas possible d'utiliser l'adressage relatif pour accéder à une ressource tel que l'affichage d'un menu lorsque cet appel peut se faire à différents endroits de l'arborescence du site.

Le fichier autoload.php active le chargement dynamique des classes évite au développeur de devoir charger lui-même au début des scripts, les classes dont il a besoin.
Les classes seront chargées à la construction de l'objet et plus au début du script.
Cela évite le chargement de classes non utilisées ou le double chargement en cas d'inclusion de script

Le pied de page est par défaut masqué dans la feuille de style, il est rendu visible par une instruction JavaScript après le remplissage de la page afin d'éviter un effet visuel peu agréable (le pied qui s'affiche avec le contenu de la page)

4. La déclaration de type

Les déclarations de types peuvent être ajoutées aux arguments des fonctions, valeurs de retour, et, à partir de PHP 7.4.0, aux propriétés de classes. Cela permet de vérifier que les valeurs transmises ou retournées correspondent bien au type spécifié au moment de l'appel. Dans le cas contraire une TypeError est lancée.

Un problème se pose néanmoins lorsqu'un paramètre peut avoir plusieurs types. Dans ce cas depuis la version 8 de PHP il est possible d'utiliser le type mixed qui signifie 'tout type de données'

Cela revient finalement à ne pas indiquer de type.

Nouveauté aussi apportée par la version 8.0, la déclaration des types de paramètre et de valeur de retour peut désormais être marquée en tant que nullable en préfixant le nom du type avec un point d'interrogation. Ceci signifie que le type spécifié aussi bien que null peuvent être passés comme argument, ou retournés en tant que valeur, respectivement.

Dernière nouveauté, les Union Types qui permettent de déclarer plusieurs types pour les arguments, les types de retour et les propriétés de classe.

```
/**
 * @param string $login
 * @return array|null
 */
static function getMembreByLogin(string $login) : object|bool
```

5. L'accès aux données

L'accès aux données s'effectue avec l'API PDO
Tous les nouveaux modules devront passer par l'utilisation des méthodes de la classe métier assurant les accès aux données.

6. Les classes techniques

Classe	Rôle
Database	Assure la connexion à la base de données Les paramètres de connexion sont conservés dans le fichier de configuration /.config/database.php
Erreur	Propose une méthode envoyerErreur() qui va en fonction du type de l'appel du script en cours (direct, ou ajax) rediriger vers la page d'erreur ou retourner dans le format JSON le message d'erreur et son type
Input et classes dérivées	Ensemble des classes permettant de contrôler les données en fonction de leur type (texte, entier, date, liste). Possibilité de paramétrer les règles de vérification (min, max, pattern)
Journal	Permet d'enregistrer des événements dans des fichiers de journalisations On mémorise, la date et l'heure, l'événement, le script et l'adresse IP
Mail	Elle permet d'envoyer des mails simples et des mails avec pièces jointes. Elle repose sur la librairie très répandue PHPMAILER (https://github.com/PHPMailer/PHPMailer) qui a l'avantage de pouvoir facilement paramétrer le SMTP pour l'envoi, faire des mails en HTML, et inclure des pièces jointes. La classe Mail propose deux méthodes : envoyer(string $destinataire, string $sujet, string $message) envoyerAvec(string $destinataire, string $sujet, string $message, string $piece) Les paramètres de configuration (host, user, password, etc.) sont conservés dans le fichier de configuration /.config/mail.php
Select	Classe permettant de lancer des requêtes SQL de consultation.
Std	La méthode formatValide($valeur, 'format') permet de vérifier le respect des principaux formats de données à l'aide d'expressions régulières. Le paramètre 'format' codifie chaque expression régulière.
Table	Classe abstraite dont la plupart des classes métier dérivent afin de rendre totalement standard les opérations d'ajout, de modification et de suppression sur une table de la base de données possédant une clé primaire simple.

7. Les composants utilisés côté client

L'application utilisent de nombreux composants : Bootstrap, jQuery, tablesorter, autocomplete (2 versions possibles), la bibliothèque fonction.js, …
Afin de s'assurer d'utiliser toujours la dernière version de ces composants, un référentiel GitHub (https://github.com/verghote/composant.git) regroupe leur CDN ou leur dernière version (fonction.js, ckeditor).

Ce référentiel n'est pas intégré dans le projet sous la forme d'un sous-module, car les fichiers qui le composent sont directement exploitables par le navigateur. Il faut simplement les rendre accessibles sur Internet. Pour cela il suffit de les héberger.

Le service GitHub Page permettant l'hébergement de pages statiques provenant d'un référentiel a été utilisé. Les ressources sont ainsi accessibles depuis l'url https://verghote.github.io/composant/
Elle le sont aussi depuis le CDN jsDelivre de façon automatique
https://cdn.jsdelivr.net/gh/verghote/composant/

L'intégration d'un composant dans une page du projet s'effectue alors par le chargement classique de la ressource lorsqu'il s'agit d'un fichier Javascript ou d'un fichier CSS ou par la lecture de son contenu pour les fichiers HTML.

Pour utiliser Bootstrap

```
echo file_get_contents('https://cdn.jsdelivr.net/gh/verghote/composant/bootstrap.html')
```

Pour utiliser jQuery

```
echo file_get_contents('https://cdn.jsdelivr.net/gh/verghote/composant/jquery.html');
```

Pour utiliser autocomplete dans sa version Javascript (tarekraafat-autocomplete.js)

```
$head .= file_get_contents('https://verghote.github.io/composant/autocomplete2.html');
```

Pour utiliser les fonctions présentes dans la bibliothèque fonction.js

```
import { afficherErreur,  filtrerLaSaisie }
from " https://verghote.github.io/composant/fonction.js";
```

Pour utiliser la version personnalisée de Ckeditor

```
$head .= "<script src=' https://verghote.github.io/composant/ckeditor/ckeditor.js'></script>";
```

8. Les librairies chargées via composer

Cf. documentation en ligne Développent web > Outils pour développeur > composer.pdf

Le fichier composer.json contient les librairies intégrées dans le projet

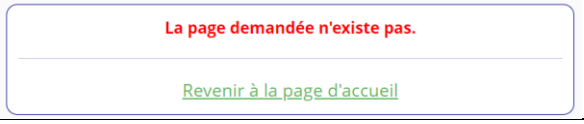
```
{
  "require": {
    "gumlet/php-image-resize": "2.0.*",
    "phpmailer/phpmailer": "^6.6",
    "ifsnop/mysqldump-php": "dev-master",
    "ext-mbstring": "*",
    "ext-pdo": "*",
    "ext-json": "*",
    "ext-curl": "*"
  }
}
```

php-image-resize permet de redimensionner une image, elle est utilisée dans la classe technique InputFileImg.
phpmailer permet l'envoie de mails.
mysqldump-php permet de sauvegarder la base de données.

9. Le traitement des erreurs

Le traitement des erreurs http (401, 403, 404 et 410) est mise en place à l'aide du fichier .htaccess qui les redirigent vers les scripts personnalisées placés dans le répertoire erreur lui-même alimenté par le référentiel GitHub https://github.com/verghote/erreur.git

```
ErrorDocument 404 /erreur/404.php
ErrorDocument 403 /erreur/403.php
ErrorDocument 401 /erreur/401.php
ErrorDocument 410 /erreur/410.php
```



Rappel : L'erreur 410 indique que la ressource demandée a été intentionnellement supprimée du serveur et n'est plus disponible

Le traitement des erreurs détectées par l'application est réalisé par l'intermédiaire de la méthode envoyerErreur de la classe technique Erreur
Elle se charge, en fonction du type d'appel (direct, ajax) de rediriger vers le script erreur/index.php ou de renvoyer l'erreur dans le format JSON.

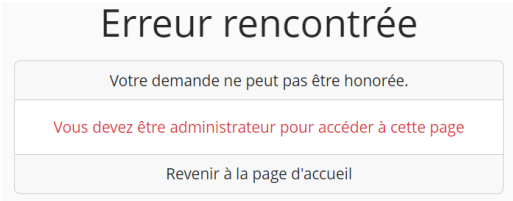
Le script erreur/index.php est chargé d'afficher l'erreur. La transmission du libellé de l'erreur s'effectue au moyen d'une variable de session 'erreur' générée au moment de la détection de l'erreur et supprimée par le script erreur/index.php après utilisation.

Exemple : un utilisateur veut atteindre une page sans en avoir les droits, le script index.php commence par vérifier le droit d'accès en appelant la méthode controlerAcces de la classe métier Administrateur :

```
Administrateur::controlerAcces();
```

La méthode va vérifier la présence de la variable de session 'membre' puis vérifier que l'id du membre fait partie des administrateurs et enfin qu'il peut administrer ce module.

```
public static function controlerAcces(): void   {
  if (!isset($_SESSION['membre'])) {
     Erreur::envoyerReponse("Vous devez être connecté pour accéder à cette fonctionnalité", 'global');
  }
  $idMembre = $_SESSION['membre']['id'];
  if (!self::estUnAdministrateur($idMembre)) {
     Erreur::envoyerReponse("Vous devez être administrateur pour accéder à cette fonctionnalité", 'global');
  }
  $repertoire = basename(dirname($_SERVER['PHP_SELF']));
  if (!self::peutAdministrer($idMembre, $repertoire)) {
     Erreur::envoyerReponse("Vous n'avez pas les droits pour accéder à cette fonctionnalité", 'global');
  }
}
```



10. Les classes métiers

La classe Administrateur

Méthode	Rôle ou requête SQL
getLesAdministrateurs	Select membre.id, concat(nom, ' ', prenom) as nom from membre join administrateur on membre.id = administrateur.id order by nom;
getLesFonctions()	Select nom, repertoire from fonction order by nom;
getLesDroits(id)	Select repertoire from droit where idAdministrateur = :id
getLesFonctionsAutorisees(id)	Select nom, fonction.repertoire from fonction join droit on fonction.repertoire = droit.repertoire where idAdministrateur = :idAdministrateur order by nom;
getLesMembres()	Select id, concat(nom, ' ', prenom) as nom from membre where id **not in (select id from administrateur)** order by nom;
estUnAdministrateur(id)	select 1 from administrateur where id = :id
peutAdministrer(id, repertoire)	select 1 from droit where idAdministrateur = :id and repertoire = :repertoire;
ajouterAdministrateur(id)	insert into administrateur (id) values (:id)
supprimerAdministrateur(id)	delete from administrateur where id = :id;
ajouterDroit(id, repertoire)	insert into droit (repertoire, idAdministrateur) values (:repertoire, :id)
supprimerDroit(id, repertoire)	delete from droit where repertoire = :repertoire and idAdministrateur = :id;
supprimerTousLesDroits(id)	delete from droit where idAdministrateur = :id;
ajouterTousLesDroits(id)	delete from droit where idAdministrateur = :id; insert into droit(idAdministrateur, repertoire) select :id, repertoire from fonction;

La classe Page

Méthode	Rôle ou requête SQL
getAll()	Select id, nom, contenu from page order by nom;
getById(id)	select contenu, nom from page where id = :id;

La classe Membre

Méthode	Rôle ou requête SQL

getAll()	Select id, concat(nom, ' ' , prenom) as nomPrenom, nom, prenom, email,
	ifnull(photo, 'Non renseignée') as photo, login,
	if(autMail, 'Oui', 'Non') as afficherMail,
	ifnull(telephone, 'Non renseigné') as telephone,
	if(SHA2('0000', 256) = password, '0000', 'Modifié') as password
	From membre
	Order by nom, prenom;

getLesMembres()	Select nom, prenom, if(autMail, email, 'Non communiqué') as mail, ifnull(telephone, 'Non renseigné') as telephone, ifnull(photo, 'Non renseignée') as photo From membre order by nom, prenom;
verifierPassword(id, password)	Select password from membre where id = :id;
getByLogin(login)	Select id, login, email, nom, prenom from membre where login = :login;
getById(id)	Select id, nom, prenom, email, login, telephone, autMail From membre where id = :id;
getPhoto(id)	Select photo from membre where id = :id; ⇨ retourne le chemin et le nom de la photo ou null si elle n'existe pas
modifierPhoto(id, nomFichier)	update membre set photo = :nomFichier where id = :id
supprimerPhoto(id)	update membre set photo = null where id = :id;
modifierPassword(login, password)	Update membre Set password = :password Where login = :login; ⇨ le password est chiffré en sha2 dans la méthode
modifierEmail(id, email)	Update membre Set email = :email Where id = :id;
modifierProfil(id, telephone, autorisation)	Construction dynamique de la requête update pour utiliser le mot clé null si la valeur est vide.

La classe Epreuve

Méthode	Rôle ou requête SQL
getAll()	Select id, date, saison, description, urlInscription, urlInscrit, dateOuverture, dateFermeture From epreuve Order by date desc;
getProchaineEpreuve()	Select saison, date_format(date, '%d/%m/%Y') as dateFr, date, description, date_format(dateOuverture, '%d/%m/%Y') as dateOuvertureFr, date_format(dateFermeture, '%d/%m/%Y') as dateFermetureFr, dateOuverture, dateFermeture, urlInscription, urlInscrit, current_date as dateJour From epreuve where date >= curdate() order by date limit 1;

11. Contrôle d'accès sur le répertoire backoffice

Le répertoire backoffice contient tous les modules d'administration du site accessible uniquement par un administrateur.

Ce répertoire ne contient pour l'instant qu'un seul module 'administrateur' qui permet de définir parmi les membres ceux qui doivent avoir des fonctions d'administration

D'autres modules seront par la suite ajoutés :

- sauvegarde Gestion des sauvegardes de la base de données
- statpage Affichage des statistiques sur les demandes de chaque page du site
- statvisite Affichage des statistiques sur les visites journalières
- journaux Affichage des fichiers de journalisations :

Le répertoire backoffice comporte son propre répertoire include contenant les fichiers suivants :

- header.thml Entête pour toutes les pages des modules
- interface.php Interface pour toutes les pages des modules (Particularité : pas de pied de page, pas de popover ni d'infobulle)

L'accès à ce répertoire est protégé par un fichier .htaccess associé à un fichier .htpasswd

Documentation en ligne : Développement Web > Protection des accès > Protection de l'accès par un fichier htaccess

Contenu du fichier .htaccess :

```
AuthUserFile j:/ppe/backoffice/.htpasswd
AuthName "Accès administration"
AuthType Basic
require user admin

# protection du fichier .htpasswd
<Files ".htpasswd">
   Require all denied
</Files>
```

Le fichier .htpasswd contient la liste des utilisateurs autorisés, ici un seul.

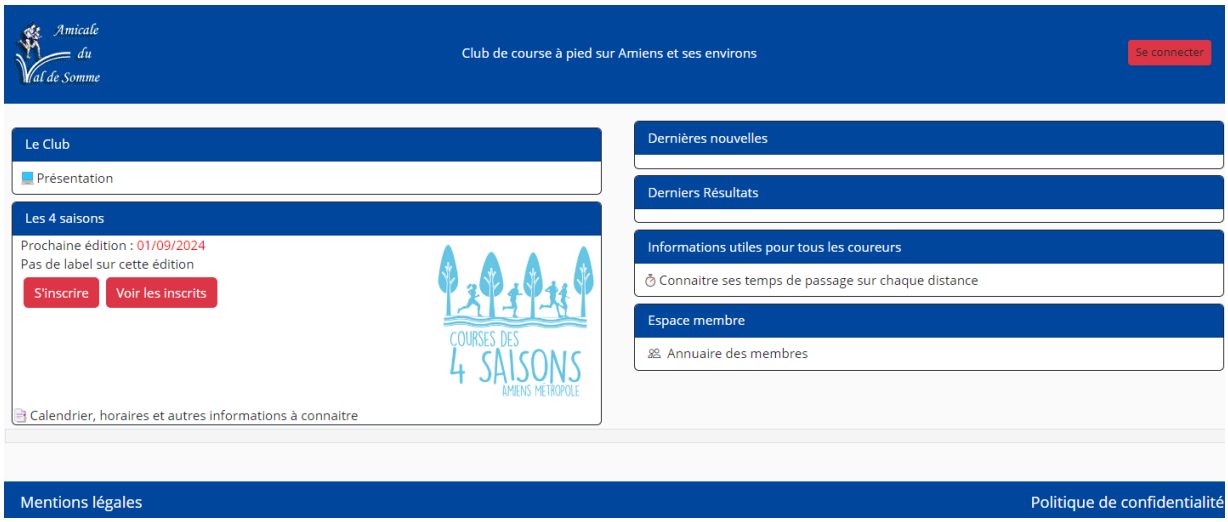
```
admin :admin
```

Nous sommes sur un serveur Windows, nous pouvons donc laisser les mots de passe en clair et créer ce fichier à l'aide d'un simple éditeur de texte.

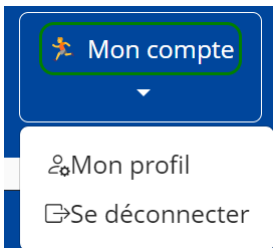
12. La page d'accueil

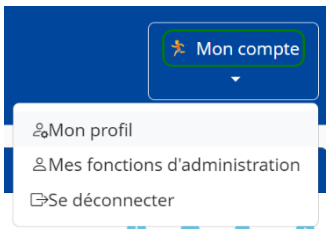
Scripts: index.php, index.js et index.html

La page d'accueil contient les points d'entrée de chaque fonctionnalité, organisés dans des cadres.

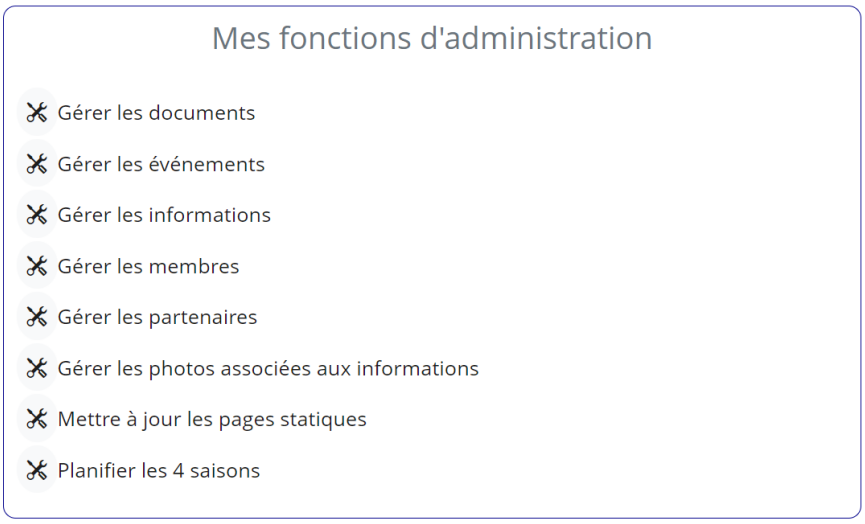


Il faut être connecté pour accéder aux fonctionnalités du cadre 'Espace membre'
Une fois connecté le bouton 'Se connecter' laisse place au bouton 'mon compte'





Pour un membre connecté disposant de droits d'administration, une option 'Mes fonctions d'administration apparaît en plus. Elle accède à la page /administration/



13. Les pages consultables par tous les visiteurs.

Actuellement, il s'agit essentiellement de pages dont le contenu HTML est sauvegardé dans la table page : Présentation (club), Calendrier, horaire et autres informations à connaitre (4 saisons), mentions légales et politique de confidentialité.

Les scripts de ces fonctionnalités sont respectivement stockés dans les répertoires club, 4saisons, mentions et politique.

Le contenu de ses 4 modules est pratiquement identique, seule la valeur de la variable $id change au niveau du fichier index.php. Ils font appel à la méthode getPage() de la classe Page pour récupérer le contenu de la page.

Le cadre 'Information utiles pour tous les coureurs' offre pour l'instant une seule fonctionnalité : 'Connaitre ses temps de passe sur chaque distance' dont les sripts sont stockés dans le répertoire vma.

Distance	Temps	Distance	Temps	Distance	Temps
1 Km	00:04:15	22 Km	01:33:30	3 Km	00:12:45
2 Km	00:08:30	23 Km	01:37:45	5 Km	00:21:15
3 Km	00:12:45	24 Km	01:42:00	10 Km	00:42:30
4 Km	00:17:00	25 Km	01:46:15	15 Km	01:03:45

Le visiteur peut indiquer son temps de passage au kilomètre pour en déduire le temps de passage sur chaque kilomètre.

14. Contrôle d'accès sur les fonctionnalités accessibles uniquement par les membres

Les fonctionnalités définies dans le cadre 'Espace membre' sont réservées aux membres du club.
Les fonctionnalités d'administration sont réservées à certains membres.

Pour les fonctionnalités réservées aux membres l'appel de la méthode controlerAcces de la classe Membre est inclus au début de chaque Endpoint

```
public static function controlerAcces(): void  {
        if (!isset($_SESSION['membre'])) {
            $_SESSION['url'] = $_SERVER['PHP_SELF'];
            header("location:/membre/connexion/");
            exit;
        }
}
```

Si la variable de session créée lors de la connexion n'existe pas, l'utilisateur est redirigé vers la page de connexion. L'url qu'il souhaitait atteindre est conservée dans une variable de session afin de pouvoir le rediriger automatiquement après la phase de connexion.

15. Contrôle d'accès sur les fonctionnalités d'administration

Il existe plusieurs solutions pour protéger l'accès aux modules d'administration

Voir documentation en ligne : Développement Web > Protection des accès > Principe du contrôle des accès par programmation

La structure de données utilisées ici correspond à la mise en place d'une protection basée sur des fonctions (répertoire) et des droits sur ces fonctions. Il suffit donc d'appliquer le principe correspondant.

L'appel de la méthode controlerAcces de la classe Administrateur est inclus au début de chaque Endpoint

```
public static function controlerAcces(): void {
    if (!isset($_SESSION['membre'])) {
        Erreur::envoyerReponse("Vous devez être connecté pour accéder à cette fonctionnalité", 'global');
    }
    $idMembre = $_SESSION['membre']['id'];
    if (!self::estUnAdministrateur($idMembre)) {
        Erreur::envoyerReponse("Vous devez être administrateur pour accéder à cette fonctionnalité", 'global');
    }
    $repertoire = basename(dirname($_SERVER['PHP_SELF']));
    if (!self::peutAdministrer($idMembre, $repertoire)) {
        Erreur::envoyerReponse("Vous n'avez pas les droits pour accéder à cette fonctionnalité", 'global');
    }
}
```

La méthode commence par vérifier que l'utilisateur est connecté et qu'il s'agit d'un administrateur (utilisation de la table administrateur)
Elle vérifie ensuite que le répertoire courant fait partie des répertoires accessibles par l'administrateur (utilisation de la table droit)

Les autres méthodes de la classe sont utilisées

```
public static function estUnAdministrateur($id) {
        $sql = "select 1 from administrateur where id = :id";
        $select = new Select();
        $reponse =  $select->getValue($sql, array("id" => $id));
        return $reponse ?? 0;
}
```

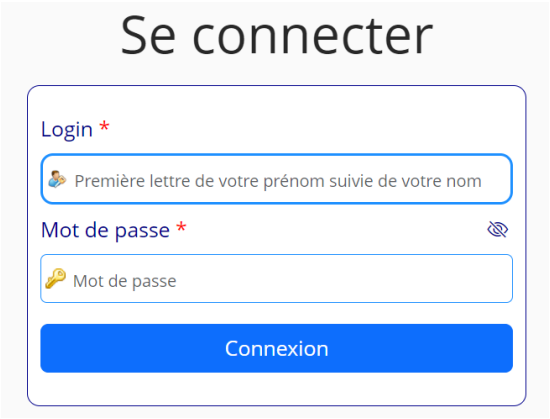
```
public static function peutAdministrer($id , $repertoire)  {
        $sql = <<<EOD
                select 1 from droit
                where idAdministrateur = :id
                and repertoire = :repertoire limit 1
EOD;
       $select = new Select();
        $reponse =  $select->getValue($sql, ["id" => $id, "repertoire" => $repertoire]);
        return $reponse ?? false;
}
```

La protection sur le script administration/index.php qui affiche les fonctions accessibles est directement intégrée dans le fichier index.php :

```
// Contrôle de l'accès : il faut être connecté et être un administrateur
if (!isset($_SESSION['membre']) ||
!Administrateur::estUnAdministrateur($_SESSION['membre']['id'])) {
    Erreur::envoyerReponse("Vous devez être administrateur pour accéder à cette page", 'global');
}
```

16. La connexion

Répertoire : membre/connexion



Les icônes placées devant les champs sont obtenues par l'attribution de la classe CSS login et de la classe password

Ces classes utilisent l'attribut background-image pour définir l'image à utiliser.
Cette image est dimensionnée en 16 * 16 sans répétition et sa position est définie pour apparaitre au début (1% 50 %)
Une marge gauche est ajoutée afin que le texte saisi ne recouvre pas l'image.

Le script index.js réalise après un contrôle (required uniquement ici) l'appel ajax vers le script ajax/connecter.php et en cas de succès redirige le visiteur vers la page retournée dans la réponse.

Le traitement côté client est classique. Une particularité repose sur l'attribution d'un événement permettant de déclencher la validation du formulaire sur la touche Entrée

```
document.addEventListener('keydown', function(event) {
    if (event.key === 'Enter') {
        if(donneesValides()) {
            connecter();
        }
    }
});
```

Une autre particularité consiste à pouvoir afficher le mot de passe en cliquant sur l'œil.

```
// sur le clic de l'oeil, on affiche le mot de passe
passwordImg.addEventListener('click', function() {
    password.type = 'text';
    passwordImg.classList.add("bi-eye");
    passwordImg.classList.remove("bi-eye-slash");

});

// quand la souris n'est plus sur l'oeil, on cache le mot de passe
passwordImg.addEventListener('mouseout', function() {
    password.type = 'password';
    passwordImg.classList.remove("bi-eye");
    passwordImg.classList.add("bi-eye-slash");
});
```

Au niveau de la sécurité, le script index.php transmet le token d'autorisation côté client.

```
global $token;
require RACINE . '/include/creationjeton.php';
```

Il redirige le membre vers la page d'accueil du site si ce dernier est déjà connecté.

```
// Si l'utilisateur est déja connecté, on le redirige vers son profil
if (isset($_SESSION['membre'])) {
    header("location:/");
    exit;
}
```

Le script ajax/connecter.php vérifie la transmission des données attendues : login et password.

```
if (!Std::existe('login', 'password' ) {
    Erreur::envoyerReponse("Tous les paramètres attendus n'ont pas été transmis", 'global');
}
```

Nous vérifions ensuite le login au niveau du format puis de son existence.

```
// vérification du login
if (!preg_match('/^[a-z]{2,}$/', $login)) {
    Erreur::envoyerReponse('Nom d'utilisateur et/ou mot de passe incorrect.', 'global');
} else {
    $membre = Membre::getByLogin($login);
    if (!$membre) {
        Erreur::envoyerReponse('Nom d'utilisateur et/ou mot de passe incorrect.', 'global');
    }
}
```

Il faut ensuite vérifier le mot de passe

```
if (!Membre::verifierPassword($membre['id'], $password)) {
        Erreur::envoyerReponse('Nom d'utilisateur et/ou mot de passe incorrect.', 'global');
}
```

Puis conserver dans une variable de session les informations du membre

```
// création de la variable de session
$_SESSION['membre'] = $membre;
```

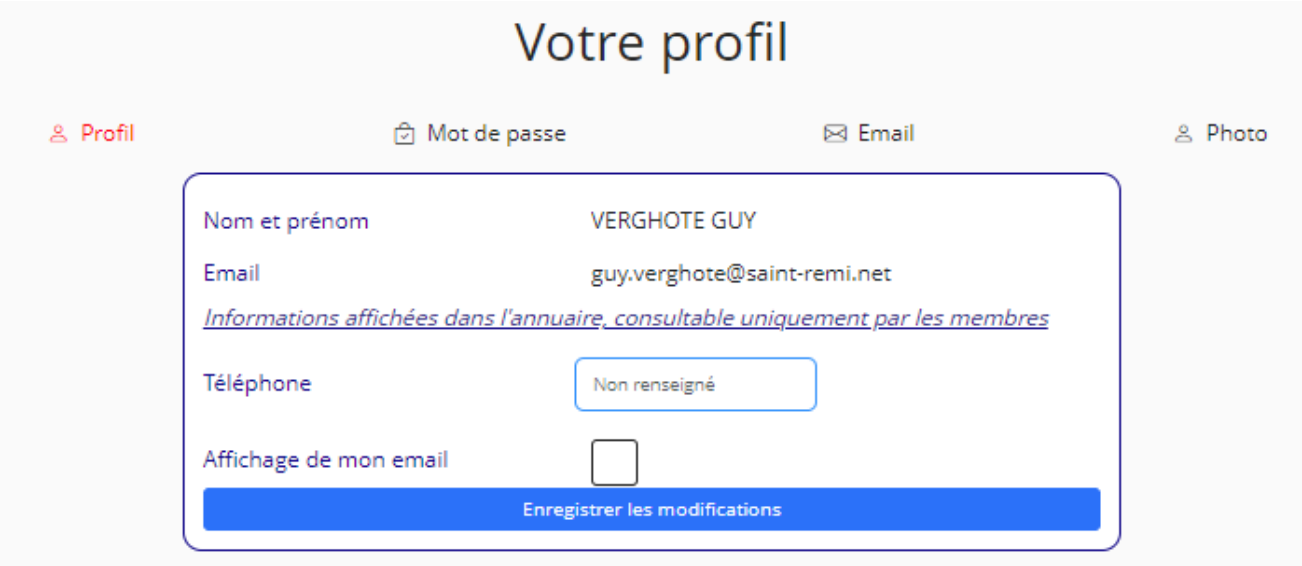
La réponse en cas de succès retourne l'url vers laquelle il faut rediriger le visiteur.

```
// Vers quelle page faut-il être redirigé ?
if(isset($_SESSION['url'])) {
    $url = $_SESSION['url'];
    unset($_SESSION['url']);
} else {
    $url = '/';
}
```

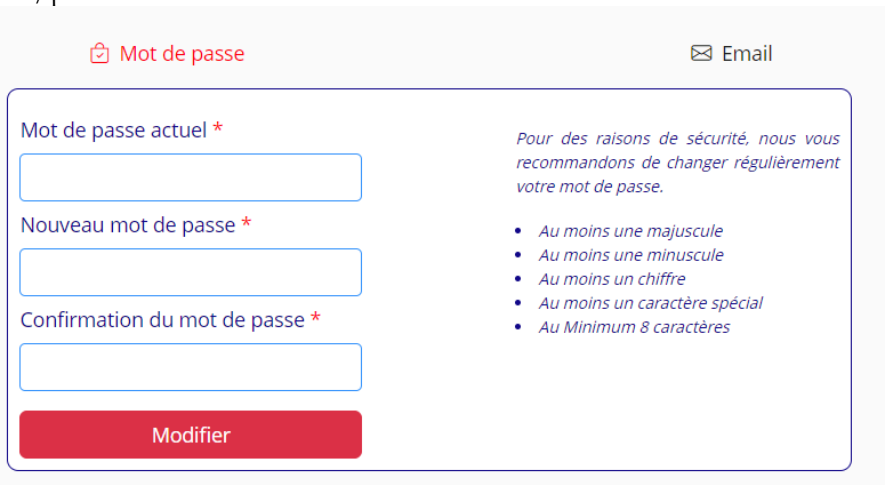
17. La modification des informations du profil

Répertoire : membre/profil

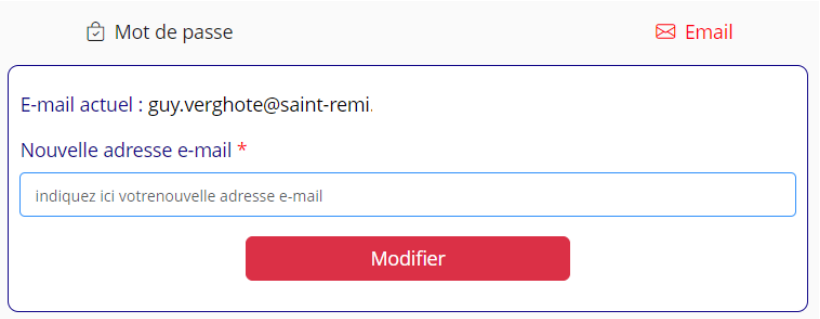
Cette fonctionnalité permet à chaque membre d'indiquer son téléphone, d'accepter ou de refuser que son mail soit affiché dans l'annuaire, de modifier son email, de modifier son mot de passe et d'ajouter une photo dans les dimensions demandées.
La photo et le téléphone seront automatiquement affichés dans l'annuaire s'ils sont renseignés par le membre.



Répertoire membre/password



Répertoire membre/email



Répertoire membre/photo

Le composant php-image-resize (https://github.com/gumlet/php-image-resize) a été ajouté afin de permettre de redimensionner les photos téléversées dans un format 200 * 200

Les paramètres à respecter pour la photo sont définies dans le fichier de configuration classemetier/configmembre.php

```
return [
    'repertoire' => '/data/photomembre',
    'extensions' => ["jpg", "png"],
    'types' => ["image/pjpeg", "image/jpeg", "x-png", "image/png"],
    'maxSize' => 200 * 1024,
    'require' => false,
    'rename' => true,
    'sansAccent' => true,
    'redimensionner' => true,
    'height' => 200,
    'width' => 200,
    'accept' => '.jpg, .png',
];
```

Toutes ces fonctionnalités ne peuvent pas utiliser les scripts standards de mise à jour car il faut pour le changement de mot de passe faire un contrôle préalable sur l'ancien mot de passe, pour la modification de l'email utiliser la variable de session pour connaitre l'identifiant du membre.

18. L'annuaire des membres

Répertoire : membre/annuaire

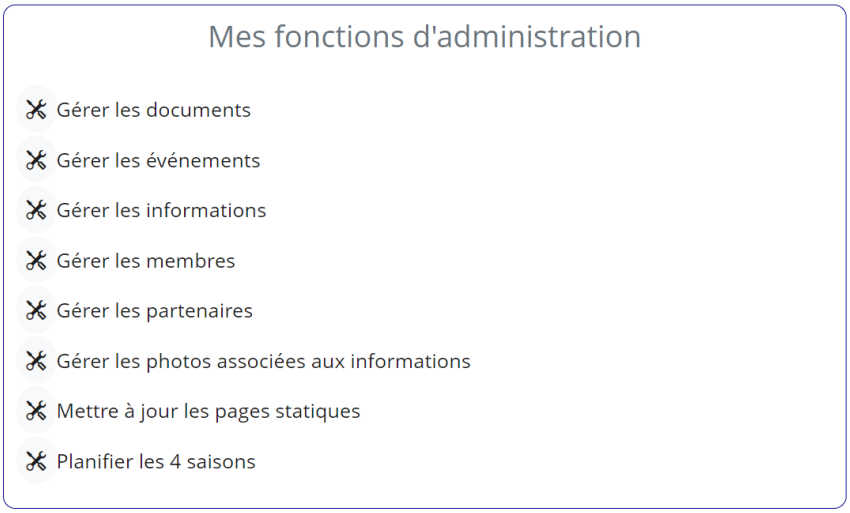
La requête source utilise les fonctions if et ifnull pour mettre en forme certaines informations

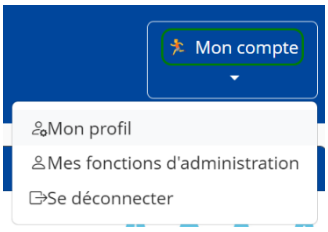
```
Select nom, prenom,
    if(autMail, email, 'Non communiqué') as mail,
    ifnull(telephone, 'Non renseigné') as telephone,
    ifnull(photo, 'Non renseignée') as photo
    From membre
    order by nom, prenom;
```

Les photos sont stockées dans le répertoire data/photomembre.

19. Les modules d'administration

Pour un membre connecté disposant de droits d'administration, une option 'Mes fonctions d'administration apparaît en plus. Elle accède à la page http://ppe/administration/





Actuellement seuls les modules suivants sont réalisés : membre, pages statiques, 4 saisons (epreuve) et les photos associées aux informations

20. La gestion des pages de présentation

Répertoire : administration/page
Script : page/index.php – index.js – index.html

Cette fonction permet de modifier les pages initialement 'statiques' du site

La page est sélectionnée à l'aide d'une zone de liste. Le contenu s'affiche dans un champ de type textarea qui est masqué et remplacé par un champ géré par le composant ckEditor. Ce composant offre des icônes permettant la mise en forme du texte en mode 'wysiwyg'.

Sa mise en place reste assez simple :

Chargement du composant dans le fichier index.php

```
$head .= "<script
src='https://cdn.jsdelivr.net/gh/verghote/composant/ckeditor/ckeditor.js'></script>";
```

Instanciation du composant pour l'associer au champ textarea dans le fichier index.js

```
CKEDITOR.replace('contenu', {height: '700px'});
```

Alimentation du composant

```
CKEDITOR.instances.contenu.setData(lesPages[page.selectedIndex].contenu);
```

Récupération de la valeur dans le composant

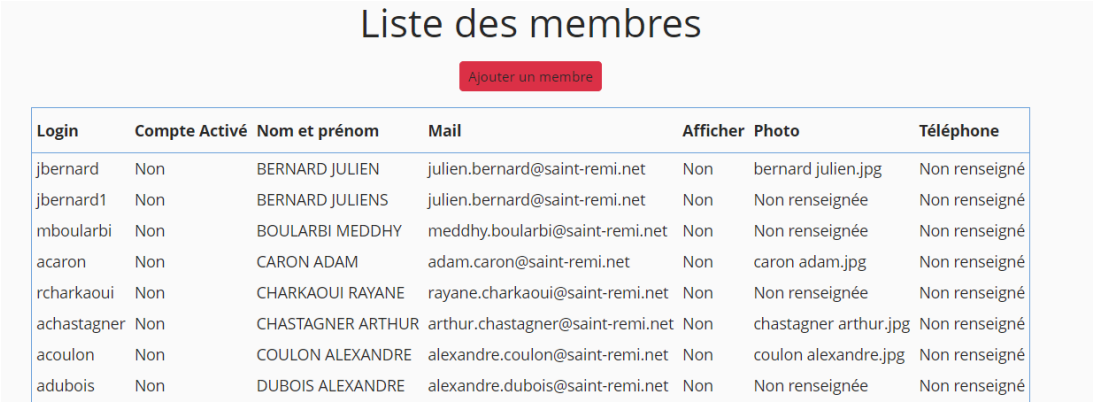
```
contenu.value = CKEDITOR.instances.contenu.getData();
```

Pour une grande majorité de ces pages l'administrateur travaille avec un document préparé sous Word qu'il peut ensuite copie/coller directement dans le champ de saisi.

Les pages statiques sont actuellement au nombre de 4 : Présentation du club, Présentation 4 saisons, Mentions légales et Politique de confidentialité

21. La gestion des membres

Répertoire : administration/membre



Liste des membres

Ajouter un membre

Login	Compte Activé	Nom et prénom	Mail	Afficher	Photo	Téléphone
jbernard	Non	BERNARD JULIEN	julien.bernard@saint-remi.net	Non	bernard julien.jpg	Non renseigné
jbernard1	Non	BERNARD JULIENS	julien.bernard@saint-remi.net	Non	Non renseignée	Non renseigné
mboularbi	Non	BOULARBI MEDDHY	meddhy.boularbi@saint-remi.net	Non	Non renseignée	Non renseigné
acaron	Non	CARON ADAM	adam.caron@saint-remi.net	Non	caron adam.jpg	Non renseigné
rcharkaoui	Non	CHARKAOUI RAYANE	rayane.charkaoui@saint-remi.net	Non	Non renseignée	Non renseigné
achastagner	Non	CHASTAGNER ARTHUR	arthur.chastagner@saint-remi.net	Non	chastagner arthur.jpg	Non renseigné
acoulon	Non	COULON ALEXANDRE	alexandre.coulon@saint-remi.net	Non	coulon alexandre.jpg	Non renseigné
adubois	Non	DUBOIS ALEXANDRE	alexandre.dubois@saint-remi.net	Non	Non renseignée	Non renseigné

Un administrateur peut simplement ajouter un membre.
La modification du login, du nom et du prénom n'est pas possible ni dans l'application, ni dans la base de données (mise en place par un trigger)
Les autres informations sont modifiables par le membre lui-même.

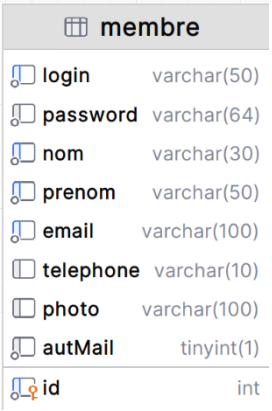
22. L'ajout d'un membre

Répertoire : administration/membre

Cette fonctionnalité permet d'ajouter un membre en indiquant uniquement le nom, le prénom et l'adresse mail.

L'ajout utilise le script standard /ajax/ajouter.php pour réaliser automatiquement l'ajout dans la table membre.

Il pourrait être judicieux d'envoyer un mail lors de la création d'un membre. Il faudrait dans ce cas écrire une version personnalisée du script ajouter.php afin d'y intégrer cet envoi.



membre	
login	varchar(50)
password	varchar(64)
nom	varchar(30)
prenom	varchar(50)
email	varchar(100)
telephone	varchar(10)
photo	varchar(100)
autMail	tinyint(1)
id	int

Le déclencheur avant ajout permet d'alimenter automatiquement les champs suivants :
login : première lettre du prénom suivie du nom
password : 0000 (il devra être personnaliser dès la première connexion de l'utilisateur)
Les champs telephone et photo seront éventuellement complétés par le membre par la suite. Si ces champs sont complétés, ils seront affichés dans l'annuaire des membres consultable uniquement par les membres.
Le champs autMail prend sa valeur par défaut : false
Ce champ permet à l'utilisateur d'autoriser l'affichage de son email dans l'annuaire des membres.

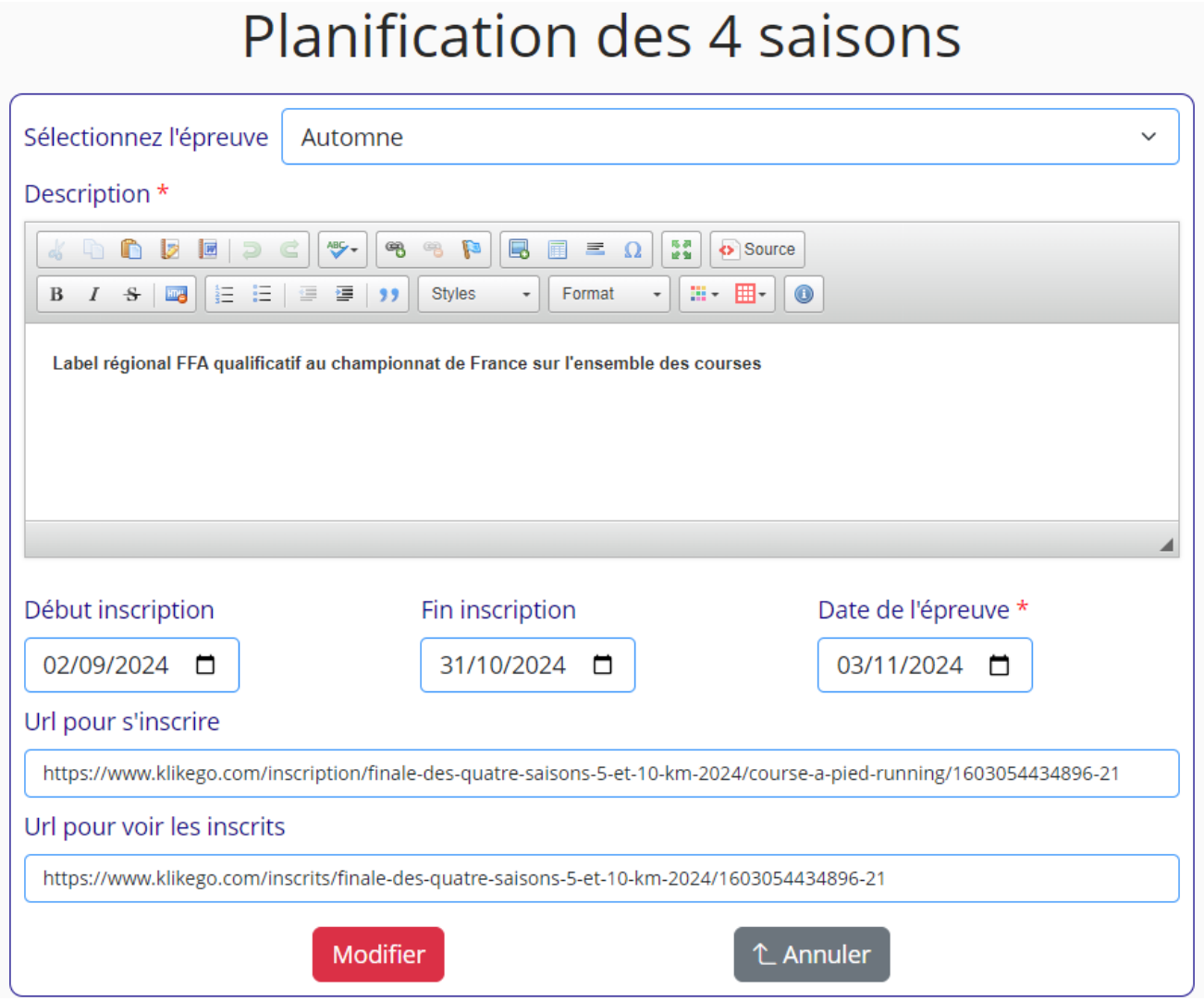
23. La planification des 4 saisons

Répertoire : administration/epreuve

Le club organise chaque année depuis 1983 4 éditions de sa course 'les 4 saisons', une par saison.

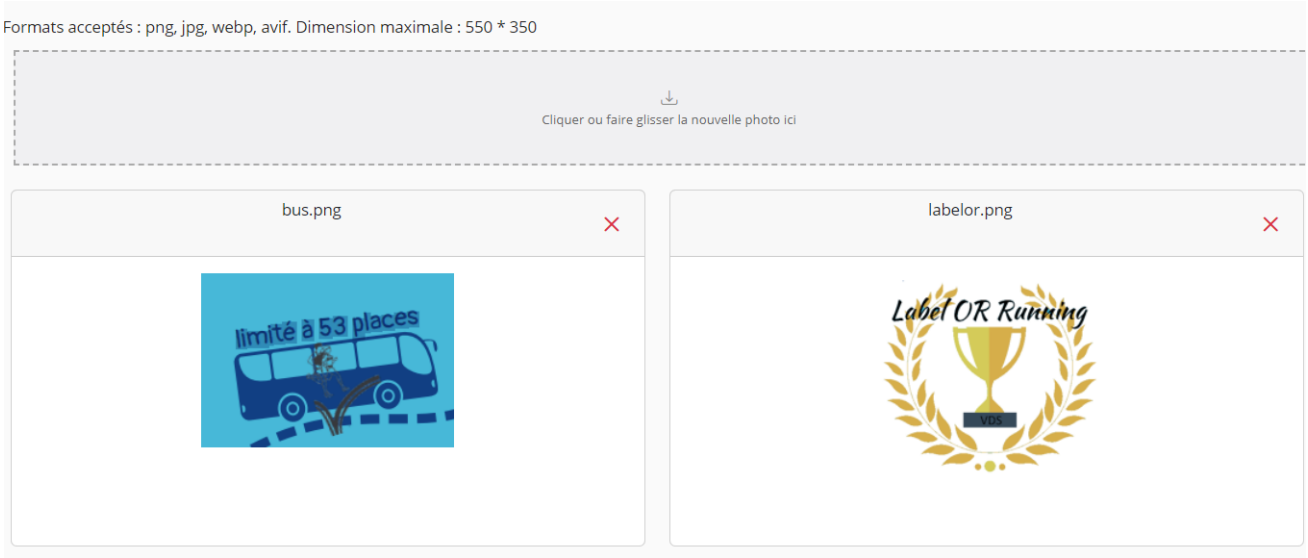
Cette fonctionnalité permet de modifier la description, la planification, et les liens pour les inscriptions en lignes.

Elle utilise la classe métier Epreuve et le script standard /ajax/modifier.php pour enregistrer les modifications dans la table 'epreuve'



24. La photothèque

Ce module permet simplement l'ajout ou la suppression d'image qui serviront à alimenter les informations publiées sur le site.



Structure du module	Commentaire
photoinformation ajax ajouter.php supprimer.php index.html index.js index.php	Le fichier /.config/photo.php contient les paramètres que doivent vérifier les fichiers téléversés Le module utilise un objet de la classe InpufFileImg afin d'automatiser les contrôles et la copie du fichier téléversé, voir sa suppression.

La suppression nécessite de passer le nom du fichier à supprimer. Il faut donc vérifier la valeur transmise afin d'éviter une attaque par injection de chemin : l'attaquant envoie '../../../../etc/passwd' afin de supprimer le fichier passwd.

Le fichier à supprimer dans se trouver dans le répertoire /data/photoinformation, il suffit de regarder si le nom complet du fichier (chemin + nom) commence par les même caractères.

```
// Vérification que le fichier est bien dans le répertoire autorisé
if (strpos(realpath($fichier), realpath(RACINE . $lesParametres['repertoire'])) !== 0) {
    Erreur::envoyerReponse("Tentative d'attaque par injection de chemin", 'global');
}
```

Une autre solution consise à vérifier que le fichier ne comporte pas de / ou de \

```
if (preg_match('/[\\/]/', $nomFichier)) {… }
```

25. La partie backoffice

Répertoire : backoffice

L'ensemble des modules présents dans ce répertoire sont accessibles uniquement par l'administrateur du site.

L'accès est protégé par un fichier .htaccess associé à un fichier .htpasswd

Ces modules ont leur propre interface (interface.php) contenu dans le sous répertoire include.
L'entête contenu dans le fichier header.html contient un menu de 5 liens pointant les 5 fonctionnalités attendues :

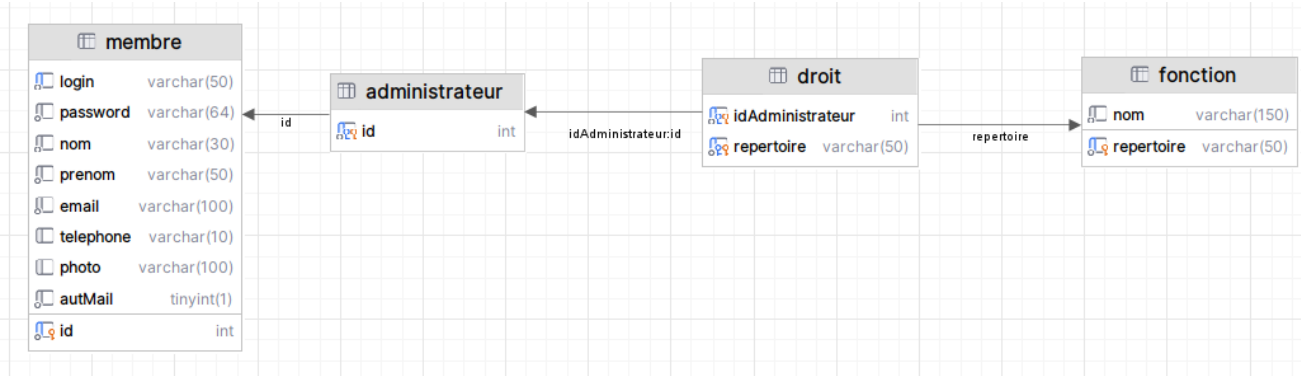
- ✓ L'attribution des droits d'administration
- ✓ La sauvegarde de la base de données
- ✓ Les statistiques de visites journalières
- ✓ Les statistiques des pages visitées
- ✓ La traçabilité des requêtes dans différents journaux

Actuellement seule l'attribution des droits d'administration est réalisée.
L'interface de gestion des administrateurs et de leurs droits a été réalisée par Jérémy Dawagne.
Son descriptif technique est fourni dans le document 'Droit d'administration.pdf'

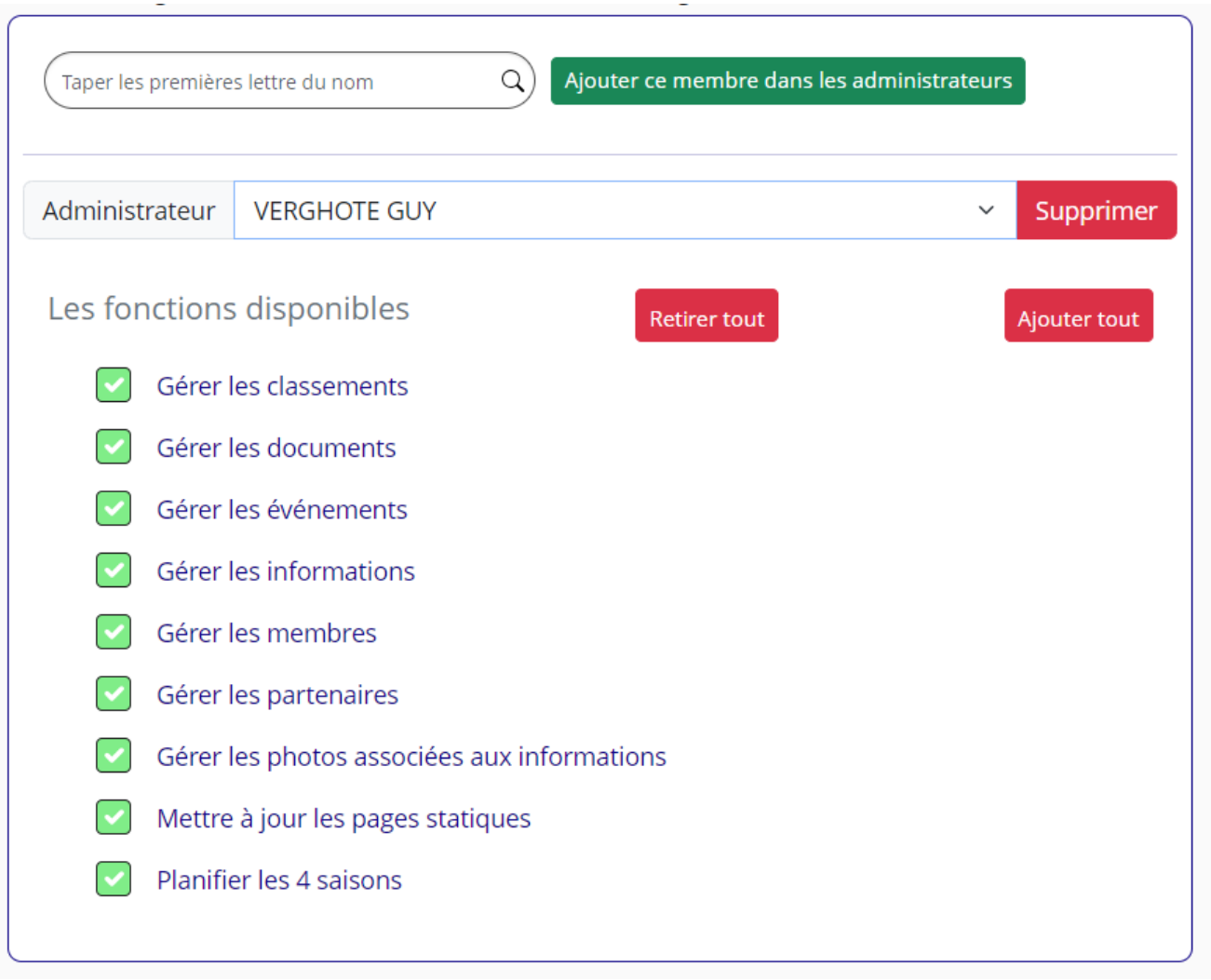
26. Gestion des droits des administrateurs

Répertoire : backoffice/administrateur

L'ensemble des informations concernant les membres de l'association sont stockées dans la table membre.
Parmi ces membres certains peuvent être autorisés à réaliser certaines fonctions administratives contenu dans la table fonction.
Les membres 'administrateur' sont stockés dans la table administrateur.
Les droits des administrateurs sont stockés dans la table droit.



Au niveau physique, une fonction correspond à un répertoire.
Par exemple, les fichiers permettant de gérer les pages statiques se trouvent dans répertoire page.
Tous ces répertoires sont placés dans le répertoire administration du site.



L'utilisateur sélectionne un administrateur dans la liste puis coche les fonctionnalités autorisées pour cet administrateur.

La mise à jour dans la base de données est déclenchée immédiatement (clic sur la case).

L'entête permet de sélectionner un membre par un système d'autocomplétion parmi les membres qui ne sont pas encore administrateur afin de l'ajouter dans la table administrateur.